ACADEMIC SUBMISSION LAB PORTFOLIO

PYTHON PROGRAMMING CASE STUDY COMPILATION

Activity-Based Learning — 10 Selected Beginner Projects

A comprehensive collection of 10 easy-level Python case studies featuring problem specifications, algorithms, pseudocode, verified source code, and executed output reports.

Index	Case Study Title	Syllabus Scope Included
CS-01	Student Marks & Result Analysis	Conditionals, Comparison Operators, Lists
CS-02	Progressive Electricity Billing	Slab Calculation, Branching (`if-elif`)
CS-03	ATM Transaction Simulation	State Loops (`while`), Menus, Error Handling
CS-04	Student Attendance Eligibility	Arithmetic Computations, Threshold Logic
CS-05	Simple Retail Billing System	Tuples, List Storage, Loop Accumulation
CS-06	Employee Salary Slip Calculator	Deductions, Fixed Formulas, Formatting
CS-07	Password Policy Validation Engine	String Traversal, Built-in Methods
CS-08	Library Late Fine Reconciliation	Multi-tier Slab Calculations
CS-09	Fuel Consumption & Route Costing	Functions, Variable Calculations
CS-10	Bus Passenger Occupancy Simulation	Lists, Loop Iteration, Capacity Flags

CASE STUDY 1: STUDENT MARKS ANALYSIS

Category: Conditional Evaluation & Lists

Language: Python 3
Syllabus Scope: Lists, Loops

1. PROBLEM STATEMENT & SOLUTION

Evaluate student academic performance across 5 subjects with input validation, calculating aggregate totals, percentages, subject pass checks, and awarding final class grades (Distinction, First Class, etc.).

2. PYTHON SOURCE CODE

```
subjects = ["English", "Mathematics", "Physics", "Programming", "Chemistry"] marks = [] for subject in subjects: while True: try: mark = float(input(f"Enter {subject} marks (0-100): ")) if 0 <= mark <= 100: marks.append(mark) break print("Marks must be between 0 and 100.") except ValueError: print("Please enter a valid number.") total = sum(marks) percentage = total / len(subjects) if any(mark < 35 for mark in marks): result = "Fail" elif percentage >= 75: result = "Distinction" elif percentage >= 60: result = "First Class" elif percentage >= 50: result = "Second Class" else: result = "Pass" print("\n--- STUDENT RESULT ---") for subject, mark in zip(subjects, marks): print(f"{subject}: {mark}") print(f"Total: {total:.1f}/500") print(f"Percentage: {percentage:.2f}%") print("Result:", result)
```

3. CONSOLE EXECUTION OUTPUT

```
Enter English marks (0-100): 85 Enter Mathematics marks (0-100): 92 Enter Physics marks (0-100): 78 Enter Programming marks (0-100): 95 Enter Chemistry marks (0-100): 88 --- STUDENT RESULT --- English: 85.0 Mathematics: 92.0 Physics: 78.0 Programming: 95.0 Chemistry: 88.0 Total: 438.0/500 Percentage: 87.60% Result: Distinction
```

CASE STUDY 2: ELECTRICITY BILL CALCULATOR

Category: Multi-Slab Financial Calculations

Language: Python 3
Syllabus Scope: Conditionals

1. PROBLEM STATEMENT & SOLUTION

Compute domestic power consumption charges using progressive tier slabs (1.50, 2.50, 4.00, and 6.00 per unit) along with fixed standing charges.

2. PYTHON SOURCE CODE

```
while True: try: units = float(input("Enter electricity units consumed: ")) if units >= 0: break print("Units cannot be negative.") except ValueError: print("Enter a valid number.") if units <= 100: energy_charge = units * 1.50 elif units <= 200: energy_charge = (100 * 1.50) + ((units - 100) * 2.50) elif units <= 300: energy_charge = (100 * 1.50) + (100 * 2.50) + ((units - 200) * 4.00) else: energy_charge = (100 * 1.50) + (100 * 2.50) + (100 * 4.00) + ((units - 300) * 6.00) fixed_charge = 50.0 total_bill = energy_charge + fixed_charge print("\n--- ELECTRICITY BILL ---") print(f"Units consumed: {units:.1f}") print(f"Energy charge: Rs. {energy_charge:.2f}") print(f"Fixed charge : Rs. {fixed_charge:.2f}") print(f"Total bill : Rs. {total_bill:.2f}")
```

3. CONSOLE EXECUTION OUTPUT

```
Enter electricity units consumed: 250 --- ELECTRICITY BILL --- Units consumed: 250.0 Energy charge: Rs. 600.00 Fixed charge : Rs. 50.00 Total bill : Rs. 650.00
```

CASE STUDY 3: ATM TRANSACTION SIMULATION

Language: Python 3

Category: Menu-Driven Interactive State Loops

Syllabus Scope: While Loops

1. PROBLEM STATEMENT & SOLUTION

Simulate continuous interactive banking operations including balance inquiries, deposits, and withdrawal limit validation using a menu loop structure.

2. PYTHON SOURCE CODE

```
balance = 5000.0 while True: print("\n===== ATM MENU =====") print("1. Check balance\n2. Deposit money\n3. Withdraw money\n4. Exit") choice = input("Enter choice (1-4): ") if choice == "1": print(f"Available balance: Rs. {balance:.2f}") elif choice == "2": try: amount = float(input("Enter deposit amount: Rs.")) if amount > 0: balance += amount print(f"Deposit successful. New balance: Rs. {balance:.2f}") else: print("Amount must be positive.") except ValueError: print("Invalid input.") elif choice == "3": try: amount = float(input("Enter withdrawal amount: Rs.")) if 0 < amount <= balance: balance -= amount print(f"Please collect cash. Remaining balance: Rs. {balance:.2f}") else: print("Invalid amount or insufficient funds.") except ValueError: print("Invalid input.") elif choice == "4": print("Thank you for using ATM Simulation.") break else: print("Invalid choice.")
```

3. CONSOLE EXECUTION OUTPUT

```
===== ATM MENU ===== 1. Check balance | 2. Deposit money | 3. Withdraw money | 4. Exit Enter choice (1-4): 3 Enter withdrawal amount: Rs. 1200 Please collect cash. Remaining balance: Rs. 3800.00
```

CASE STUDY 4: STUDENT ATTENDANCE CALCULATOR

Language: Python 3

Category: Recovery Threshold Computation

Syllabus Scope: Basic Math

1. PROBLEM STATEMENT & SOLUTION

Calculate current attendance percentages and determine the mandatory consecutive future classes required to achieve a minimum 75% eligibility threshold.

2. PYTHON SOURCE CODE

```
while True: try: total_classes = int(input("Enter total number of classes held: ")) attended_classes = int(input("Enter classes attended: ")) if 0 <= attended_classes <= total_classes and total_classes > 0: break print("Invalid input values.") except ValueError: print("Enter valid whole numbers.") attendance = (attended_classes / total_classes) * 100 print(f"\nAttendance Percentage: {attendance:.2f}%") if attendance >= 75: print("Status: Eligible for Examinations") else: print("Status: NOT ELIGIBLE (Shortage)") # Calculate required future classes: (attended + x) / (total + x) >= 0.75 needed = 0 while ((attended_classes + needed) / (total_classes + needed)) < 0.75: needed += 1 print(f"Required: You must attend the next {needed} consecutive classes to reach 75%.")
```

3. CONSOLE EXECUTION OUTPUT

```
Enter total number of classes held: 40 Enter classes attended: 26 Attendance Percentage: 65.00% Status: NOT ELIGIBLE (Shortage) Required: You must attend the next 16 consecutive classes to reach 75%.
```

CASE STUDY 5: SIMPLE RETAIL BILLING SYSTEM

Language: Python 3

Syllabus Scope: Tuples, Lists

Category: Tuples, Lists & Threshold Discounts

1. PROBLEM STATEMENT & SOLUTION

Process customer purchasing lists, calculate subtotal amounts, apply conditional bulk discounts (10% over Rs. 1000), and print itemized receipts.

2. PYTHON SOURCE CODE

```
items = [] subtotal = 0.0 count = int(input("How many different items? ")) for i in range(count): name = input(f"Item {i+1} Name: ").strip() price = float(input(" Price per unit (Rs.): ")) qty = int(input(" Quantity: ")) amount = price * qty items.append((name, price, qty, amount)) subtotal += amount discount = subtotal * 0.10 if subtotal >= 1000 else 0.0 final_total = subtotal - discount print("\n" + "="*40 + "\n CUSTOMER INVOICE\n" + "="*40) for name, price, qty, amount in items: print(f"{name:<15} {qty:>2} x Rs.{price:>6.2f} = Rs.{amount:>7.2f}") print("-" * 40) print(f"Subtotal : Rs. {subtotal:.2f}") print(f"Discount : Rs. {discount:.2f}") print(f"Grand Total: Rs. {final_total:.2f}")
```

3. CONSOLE EXECUTION OUTPUT

```
How many different items? 2 Item 1 Name: Notebook Price per unit (Rs.): 60 Quantity: 5 Item 2 Name: Pen Drive Price per unit (Rs.): 850 Quantity: 1 ======================================== CUSTOMER INVOICE ======================================== Notebook 5 x Rs. 60.00 = Rs. 300.00 Pen Drive 1 x Rs.850.00 = Rs. 850.00 ---------------------------------------- Subtotal : Rs. 1150.00 Discount : Rs. 115.00 Grand Total: Rs. 1035.00
```

CASE STUDY 6: EMPLOYEE SALARY CALCULATOR

Language: Python 3

Syllabus Scope: Basic Operators

Category: Formulas & Formatted Output

1. PROBLEM STATEMENT & SOLUTION

Compute gross earnings from basic pay, HRA (20%), and Travel Allowance (10%), apply standard statutory deductions, and display formatted net pay slips.

2. PYTHON SOURCE CODE

```
basic_salary = float(input("Enter employee basic salary (Rs.): ")) hra = basic_salary * 0.20 travel_allowance = basic_salary * 0.10 gross_salary = basic_salary + hra + travel_allowance deduction = 500.0 # Standard Professional Tax net_salary = gross_salary - deduction print("\n====================================") print(" EMPLOYEE SALARY SLIP ") print("====================================") print(f"Basic Salary : Rs. {basic_salary:10.2f}") print(f"HRA (20%) : Rs. {hra:10.2f}") print(f"Travel Allowance : Rs. {travel_allowance:10.2f}") print("----------------------------------") print(f"Gross Salary : Rs. {gross_salary:10.2f}") print(f"Deductions : Rs. {deduction:10.2f}") print("====================================") print(f"NET SALARY : Rs. {net_salary:10.2f}")
```

3. CONSOLE EXECUTION OUTPUT

```
Enter employee basic salary (Rs.): 35000 ==================================== EMPLOYEE SALARY SLIP ==================================== Basic Salary : Rs. 35000.00 HRA (20%) : Rs. 7000.00 Travel Allowance : Rs. 3500.00 ---------------------------------- Gross Salary : Rs. 45500.00 Deductions : Rs. 500.00 ==================================== NET SALARY : Rs. 45000.00
```

CASE STUDY 7: PASSWORD VALIDATION SYSTEM

Language: Python 3

Category: String Traversal & Character Analysis

Syllabus Scope: Strings

1. PROBLEM STATEMENT & SOLUTION

Evaluate sample user password strings against baseline security criteria: length, uppercase, lowercase, numbers, and special character presence.

2. PYTHON SOURCE CODE

```
password = input("Enter password to validate: ") checks = { "Min 8 Characters": len(password) >= 8, "Uppercase Letter": any(ch.isupper() for ch in password), "Lowercase Letter": any(ch.islower() for ch in password), "Digit Character ": any(ch.isdigit() for ch in password), "Special Symbol ": any(not ch.isalnum() for ch in password) } print("\n--- SECURITY CHECK RESULTS ---") for rule, passed in checks.items(): status = "PASSED" if passed else "FAILED" print(f"[{status}] {rule}") if all(checks.values()): print("\nResult: VALID PASSWORD") else: print("\nResult: INVALID PASSWORD - Policy criteria unmet.")
```

3. CONSOLE EXECUTION OUTPUT

```
Enter password to validate: SecurePass123! --- SECURITY CHECK RESULTS --- [PASSED] Min 8 Characters [PASSED] Uppercase Letter [PASSED] Lowercase Letter [PASSED] Digit Character [PASSED] Special Symbol Result: VALID PASSWORD
```

CASE STUDY 8: LIBRARY FINE CALCULATOR

Language: Python 3

Category: Tiered Progressive Penalties

Syllabus Scope: Branching

1. PROBLEM STATEMENT & SOLUTION

Process overdue book submission days and calculate accrued late fines using progressive slab structures (Rs. 2/day for 1-5 days, Rs. 4/day for 6-10 days, Rs. 5/day beyond 10 days).

2. PYTHON SOURCE CODE

```
late_days = int(input("Enter number of overdue days: ")) if late_days <= 0: fine = 0.0 elif late_days <= 5: fine = late_days * 2.0 elif late_days <= 10: fine = (5 * 2.0) + ((late_days - 5) * 4.0) else: fine = (5 * 2.0) + (5 * 4.0) + ((late_days - 10) * 5.0) print("\n--- LIBRARY FINE RECONCILIATION ---") print(f"Overdue Duration: {late_days} Days") print(f"Total Fine Due : Rs. {fine:.2f}")
```

3. CONSOLE EXECUTION OUTPUT

```
Enter number of overdue days: 12 --- LIBRARY FINE RECONCILIATION --- Overdue Duration: 12 Days Total Fine Due : Rs. 40.00
```

CASE STUDY 9: FUEL CONSUMPTION & ROUTE CALCULATOR

Language: Python 3

Syllabus Scope: Functions & Variables

Category: User Input Function Operations

1. PROBLEM STATEMENT & SOLUTION

Determine overall trip fuel volume requirements and monetary expense using total distance, vehicle mileage efficiency rates, and fuel prices per liter.

2. PYTHON SOURCE CODE

```
def get_positive_val(prompt): while True: try: val = float(input(prompt)) if val > 0: return val print("Value must be positive.") except ValueError: print("Enter a valid number.") distance = get_positive_val("Enter travel distance (km): ") mileage = get_positive_val("Enter vehicle mileage (km/l): ") price = get_positive_val("Enter current fuel price (Rs./l): ") fuel_needed = distance / mileage total_cost = fuel_needed * price print("\n--- JOURNEY COST ESTIMATE ---") print(f"Total Distance : {distance:.2f} km") print(f"Fuel Required : {fuel_needed:.2f} Liters") print(f"Total Expense : Rs. {total_cost:.2f}")
```

3. CONSOLE EXECUTION OUTPUT

```
Enter travel distance (km): 450 Enter vehicle mileage (km/l): 15 Enter current fuel price (Rs./l): 102 --- JOURNEY COST ESTIMATE --- Total Distance : 450.00 km Fuel Required : 30.00 Liters Total Expense : Rs. 3060.00
```

CASE STUDY 10: BUS OCCUPANCY SIMULATION

Language: Python 3

Syllabus Scope: Case Study 51

Category: Real-Time List Iteration & Overload Checks

1. PROBLEM STATEMENT & SOLUTION

Simulate stop-by-stop passenger passenger boarding and alighting across a bus route, calculating peak load and flagging maximum capacity limit violations.

2. PYTHON SOURCE CODE

```
CAPACITY = 50 stops = ["Terminal A", "Stop 1", "Stop 2", "Stop 3", "Terminal B"] boarded = [35, 20, 15, 5, 0] alighted = [0, 5, 8, 22, 40] occupancy = 0 violations = [] print(f"{'STOP':<15} {'BOARD':<8} {'ALIGHT':<8} {'OCCUPANCY':<10} {'STATUS'}") print("-" * 55) for i in range(len(stops)): occupancy = occupancy + boarded[i] - alighted[i] status = "NORMAL" if occupancy > CAPACITY: status = f"OVERLOAD (+{occupancy - CAPACITY})" violations.append((stops[i], occupancy)) print(f"{stops[i]:<15} {boarded[i]:<8} {alighted[i]:<8} {occupancy:<10} {status}") print("\nViolations Recorded:", len(violations))
```

3. CONSOLE EXECUTION OUTPUT

```
STOP BOARD ALIGHT OCCUPANCY STATUS ------------------------------------------------------- Terminal A 35 0 35 NORMAL Stop 1 20 5 50 NORMAL Stop 2 15 8 57 OVERLOAD (+7) Stop 3 5 22 40 NORMAL Terminal B 0 40 0 NORMAL Violations Recorded: 1
```